

Sri Lanka Institute of Information Technology



SE2020 – Web and Mobile Technologies
Year 2, Semester 2- 2026

Practical Answer Submission

Practical Sheet No: 09

Student ID	IT24102099
Student Name	Mathuppriya Naguleswaran
Campus/ Center name	SLIIT – Northern UNI
Specialization	Software Engineering
Batch No	1

MongoDB

MongoDB

- It is a NoSQL document database.
- works well with JSON-like data in Node.js.

Mongoose

- It helps define schemas, validations, and relationships more easily.

Why create separate models?

- Because Student, MenuItem, and Order are different entities in the system.

Why use references in Order?

- Because one order belongs to a student and contains menu items from another collection.

populate()

- To fetch full related document details instead of only IDs.

Why use aggregation?

- To calculate summaries such as total spent, top selling items, and daily order count efficiently in the database.

Why use pagination?

- To avoid returning too many orders at once and improve performance.

Why use regex in search?

- To support partial and case-insensitive searching.

Step 1: Create the project folder.

Step 2: Open the folder in VS Code.

Step 3: Open terminal in VS Code.

Step 4: Initialize Node project.

npm init -y

- This creates a package.json file.

```
PS C:\Users\vallu\Desktop\campus-food-api_It24102099> npm init -y
Wrote to C:\Users\vallu\Desktop\campus-food-api_It24102099\package.json:

{
  "name": "campus-food-api_it24102099",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Step 5: Install required packages

npm install express mongoose cors dotenv

- express → to create the API server
- mongoose → to connect Node.js with MongoDB
- cors → to allow cross-origin requests
- dotenv → to load values from .env

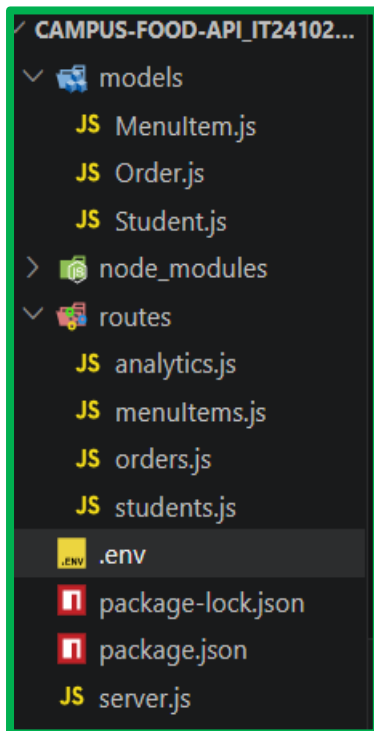
```
PS C:\Users\vallu\Desktop\campus-food-api_It24102099> npm install express mongoose cors dotenv

added 85 packages, and audited 86 packages in 14s

25 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
❖ PS C:\Users\vallu\Desktop\campus-food-api_It24102099> █
```

Step 6: Create file and folder structure



Step 7: Setup MongoDB connection

For the data base im using local MongoDB. So I don't have any password for that. If I use MongoDB Atlas **must** have database username , database password, cluster connection string.

In .env file type this,

PORT=3000

MONGO_URI=mongodb://127.0.0.1:27017/campus_food

Or

MONGO_URI=mongodb://localhost:27017/campus_food

- localhost = your own computer
- 27017 = default MongoDB port
- campus_food = database name, MongoDB will create it when data is inserted

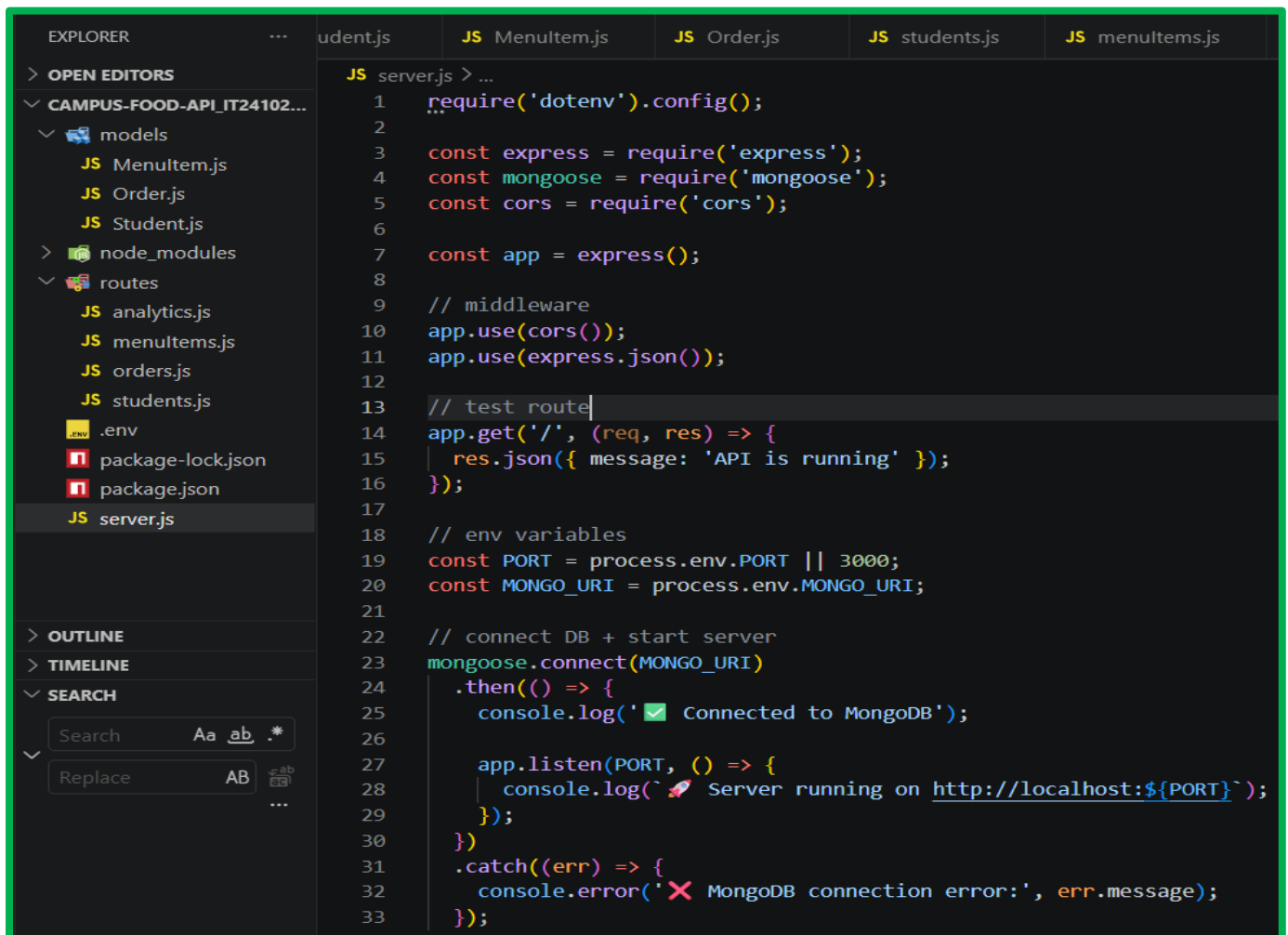
```
npm start\Users\vallu\Desktop\campus-food-api_It24102099>
> campus-food-api_it24102099@1.0.0 start
> node server.js

[dotenv@17.3.1] injecting env (2) from .env -- tip: ⚙️ suppress all logs with { quiet: true }
✅ Connected to MongoDB
🚀 Server running on http://localhost:3000
```

Step 8: Write server.js

Run the server easily with one command.

- `npm start` or `node server.js`



```
1  require('dotenv').config();
2
3  const express = require('express');
4  const mongoose = require('mongoose');
5  const cors = require('cors');
6
7  const app = express();
8
9  // middleware
10 app.use(cors());
11 app.use(express.json());
12
13 // test route
14 app.get('/', (req, res) => {
15   res.json({ message: 'API is running' });
16 });
17
18 // env variables
19 const PORT = process.env.PORT || 3000;
20 const MONGO_URI = process.env.MONGO_URI;
21
22 // connect DB + start server
23 mongoose.connect(MONGO_URI)
24   .then(() => {
25     console.log('✅ Connected to MongoDB');
26
27     app.listen(PORT, () => {
28       console.log(`🚀 Server running on http://localhost:${PORT}`);
29     });
30   })
31   .catch((err) => {
32     console.error('❌ MongoDB connection error:', err.message);
33   });
```

- `require('dotenv').config();` : Loads values from .env.
- `express` : Creates the server.
- `Mongoose` : Connects to MongoDB.
- `Cors` : Allows requests from other apps or tools.
- `app.use(express.json())` : server read JSON body data from Postman.
- `app.get('/')` : Creates a simple route to test whether server works.
- `app.use('/students', studentRoutes)` : Mounts student routes.
- `mongoose.connect(MONGO_URI)` : Connects to database.

Step 9: Add start script in package.json

When open the package.json its have this version of code

```

package.json > ...
1  {
2    "name": "campus-food-api_it24102099",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
6    > Debug
7    "scripts": {
8      "test": "echo \"Error: no test specified\" && exit 1"
9    },
10   "keywords": [],
11   "author": "",
12   "license": "ISC",
13   "type": "commonjs",
14   "dependencies": {
15     "cors": "^2.8.6",
16     "dotenv": "^17.3.1",
17     "express": "^5.2.1",
18     "mongoose": "^9.3.3"
19   }

```

But here I want to change and add extra things related to my project

My project have server.js **NOT** index.js

And add **"start": "node server.js"**

- This tells npm what to run when you type: **npm start**

```

package.json > ...
1  {
2    "name": "campus-food-api_it24102099",
3    "version": "1.0.0",
4    "description": "",
5    "main": "server.js",
6    > Debug
7    "scripts": {
8      "start": "node server.js"
9    },
10   "keywords": [],
11   "author": "",
12   "license": "ISC",
13   "type": "commonjs",
14   "dependencies": {
15     "cors": "^2.8.6",
16     "dotenv": "^17.3.1",
17     "express": "^5.2.1",
18     "mongoose": "^9.3.3"
19   }

```

If you want auto restart during development, install nodemon:

```
npm install --save-dev nodemon
```

Then use:

```
"scripts": {  
  "start": "node server.js",  
  "dev": "nodemon server.js"  
}
```

So we can start server easily with one command.

Step 10: Run the server

```
npm start\\Users\\vallu\\Desktop\\campus-food-api_It24102099>  
  
> campus-food-api_it24102099@1.0.0 start  
> node server.js  
  
[dotenv@17.3.1] injecting env (2) from .env -- tip: ⚙ suppress all logs with { quiet: true }  
✅ Connected to MongoDB  
🚀 Server running on http://localhost:3000
```

Step 11: Create the Student model

```
models > JS Studentjs > ...  
1  const mongoose = require('mongoose');  
2  
3  const studentSchema = new mongoose.Schema(  
4    {  
5      name: {  
6        type: String,  
7        required: true,  
8        trim: true  
9      },  
10     email: {  
11       type: String,  
12       required: true,  
13       unique: true,  
14       trim: true  
15     },  
16     faculty: {  
17       type: String,  
18       trim: true  
19     },  
20     year: {  
21       type: Number,  
22       min: 1,  
23       max: 4  
24     }  
25   },  
26   {  
27     timestamps: true  
28   }  
29 );  
30  
31 const Student = mongoose.model('Student', studentSchema);  
32  
33 module.exports = Student;  
34
```

- required: true => Field cannot be empty.
- unique: true =>Two students cannot have same email.
- trim: true =>Removes extra spaces.
- min: 1, max: 4 =>Year must be between 1 and 4.
- timestamps: true =>Mongoose automatically adds createdAt and updatedAt.

Step 12: Create the MenuItem model

```
models > JS MenuItem.js > ...
1  const mongoose = require('mongoose');
2
3  const menuItemSchema = new mongoose.Schema(
4    {
5      name: {
6        type: String,
7        required: true,
8        trim: true
9      },
10     price: {
11       type: Number,
12       required: true,
13       min: 0
14     },
15     category: {
16       type: String,
17       trim: true
18     },
19     isAvailable: {
20       type: Boolean,
21       default: true
22     }
23   },
24   {
25     timestamps: true
26   }
27 );
28
29 const MenuItem = mongoose.model('MenuItem', menuItemSchema);
30
31 module.exports = MenuItem;
32
```

Step 13: Create the Order model

This is the first time you are using **references**.

- student stores the ObjectId of a Student
- menuItem stores the ObjectId of a MenuItem

So Order connects all collections together.

models > JS Order.js > ...

```
1  const mongoose = require('mongoose');
2
3  const orderItemSchema = new mongoose.Schema(
4    {
5      menuItem: {
6        type: mongoose.Schema.Types.ObjectId,
7        ref: 'MenuItem',
8        required: true
9      },
10     quantity: {
11       type: Number,
12       required: true,
13       min: 1
14     }
15   },
16   {
17     _id: false
18   }
19 );
20
```

```
21 const orderSchema = new mongoose.Schema({
22   student: {
23     type: mongoose.Schema.Types.ObjectId,
24     ref: 'Student',
25     required: true
26   },
27   items: {
28     type: [orderItemSchema],
29     validate: [(val) => val.length > 0, 'At least one item is required']
30   },
31   totalPrice: {
32     type: Number,
33     required: true,
34     min: 0
35   },
36   status: {
37     type: String,
38     enum: ['PLACED', 'PREPARING', 'DELIVERED', 'CANCELLED'],
39     default: 'PLACED'
40   },
41   createdAt: {
42     type: Date,
43     default: Date.now
44   }
45 });
46
47 const Order = mongoose.model('Order', orderSchema);
48
49 module.exports = Order;
50
```

Step 14: Create student routes

```
routes > JS students.js > ...
1  const express = require('express');
2  const Student = require('../models/Student');
3
4  const router = express.Router();
5
6  // POST /students
7  router.post('/', async (req, res) => {
8    try {
9      const student = new Student(req.body);
10     const saved = await student.save();
11     res.status(201).json(saved);
12   } catch (err) {
13     console.error('Error creating student:', err.message);
14     res.status(400).json({ error: err.message });
15   }
16 });
17
18 // GET /students/:id
19 router.get('/:id', async (req, res) => {
20   try {
21     const student = await Student.findById(req.params.id);
22
23     if (!student) {
24       return res.status(404).json({ error: 'Student not found' });
25     }
26
27     res.json(student);
28   } catch (err) {
29     console.error('Error fetching student:', err.message);
30     res.status(400).json({ error: 'Invalid student ID' });
31   }
32 });
33
34 module.exports = router;
```

Step 15: Create menu item routes

The search should support partial case-insensitive name match.

{ \$regex: name, \$options: 'i' }

This means:

- search partially
- ignore uppercase/lowercase

So egg can match Egg Fried Rice.

```

routes > JS menuItems.js > ...
1  const express = require('express');
2  const MenuItem = require('../models/MenuItem');
3
4  const router = express.Router();
5
6  // POST /menu-items
7  router.post('/', async (req, res) => {
8    try {
9      const menuItem = new MenuItem(req.body);
10     const saved = await menuItem.save();
11     res.status(201).json(saved);
12   } catch (err) {
13     console.error('Error creating menu item:', err.message);
14     res.status(400).json({ error: err.message });
15   }
16 });
17
18 // GET /menu-items
19 router.get('/', async (req, res) => {
20   try {
21     const items = await MenuItem.find().sort({ createdAt: -1 });
22     res.json(items);
23   } catch (err) {
24     console.error('Error fetching menu items:', err.message);
25     res.status(500).json({ error: 'Server error' });
26   }
27 });
28

```

```

29 // GET /menu-items/search?name=...&category=...
30 router.get('/search', async (req, res) => {
31   try {
32     const { name, category } = req.query;
33
34     const filter = {};
35
36     if (name) {
37       filter.name = { $regex: name, $options: 'i' };
38     }
39
40     if (category) {
41       filter.category = category;
42     }
43
44     const items = await MenuItem.find(filter).sort({ name: 1 });
45     res.json(items);
46   } catch (err) {
47     console.error('Error searching menu items:', err.message);
48     res.status(500).json({ error: 'Server error' });
49   }
50 });
51
52 module.exports = router;

```

Step 16: Create order routes

❖ populate()

- Without populate, you only get IDs.

With populate, you get full student and menu item details.

Example:

Instead of only this:

"student": "abc123"

you get:

```
"student": {
  "_id": "abc123",
  "name": "Alice"
}
```

```
routes > JS orders.js > calculateTotalPrice
1  const express = require('express');
2  const Order = require('../models/Order');
3  const MenuItem = require('../models/MenuItem');
4  const Student = require('../models/Student');
5
6  const router = express.Router();
7
8  async function calculateTotalPrice(items) {
9    const menuItemIds = items.map(item => item.menuItem);
10
11    const menuItems = await MenuItem.find({ _id: { $in: menuItemIds } });
12
13    const priceMap = {};
14    menuItems.forEach(item => {
15      priceMap[item._id.toString()] = item.price;
16    });
17
18    let total = 0;
19
20    for (const item of items) {
21      const price = priceMap[item.menuItem.toString()];
22      if (price === undefined) {
23        throw new Error(`Invalid menuItem ID: ${item.menuItem}`);
24      }
25      total += price * item.quantity;
26    }
27
28    return total;
29  }
30
```

```

31 // POST /orders
32 router.post('/', async (req, res) => {
33   try {
34     const { student, items } = req.body;
35
36     if (!student) {
37       return res.status(400).json({ error: 'Student is required' });
38     }
39
40     const studentExists = await Student.findById(student);
41     if (!studentExists) {
42       return res.status(400).json({ error: 'Student not found' });
43     }
44
45     if (!Array.isArray(items) || items.length === 0) {
46       return res.status(400).json({ error: 'Items must be a non-empty array' });
47     }
48
49     const totalPrice = await calculateTotalPrice(items);
50
51     const order = new Order({
52       student,
53       items,
54       totalPrice,
55       status: 'PLACED'
56     });
57
58     const saved = await order.save();
59

```

```

59
60     const populatedOrder = await Order.findById(saved._id)
61       .populate('student')
62       .populate('items.menuItem');
63
64     res.status(201).json(populatedOrder);
65   } catch (err) {
66     console.error('Error placing order:', err.message);
67     res.status(400).json({ error: err.message });
68   }
69 });
70
71 // GET /orders?page=1&limit=2
72 router.get('/', async (req, res) => {
73   try {
74     const page = Number(req.query.page) || 1;
75     const limit = Number(req.query.limit) || 10;
76     const skip = (page - 1) * limit;
77
78     const orders = await Order.find()
79       .sort({ createdAt: -1 })
80       .skip(skip)
81       .limit(limit)
82       .populate('student')
83       .populate('items.menuItem');
84
85     const totalOrders = await Order.countDocuments();
86     const totalPages = Math.ceil(totalOrders / limit);
87

```

```

87
88     res.json({
89         page,
90         limit,
91         totalOrders,
92         totalPages,
93         orders
94     });
95 } catch (err) {
96     console.error('Error fetching orders:', err.message);
97     res.status(500).json({ error: 'Server error' });
98 }
99 });
100
101 // GET /orders/:id
102 router.get('/:id', async (req, res) => {
103     try {
104         const order = await Order.findById(req.params.id)
105             .populate('student')
106             .populate('items.menuItem');
107
108         if (!order) {
109             return res.status(404).json({ error: 'Order not found' });
110         }
111
112         res.json(order);
113     } catch (err) {
114         console.error('Error fetching order:', err.message);
115         res.status(400).json({ error: 'Invalid order ID' });
116     }
117 });

```

```

118
119 // PATCH /orders/:id/status
120 router.patch('/:id/status', async (req, res) => {
121     try {
122         const { status } = req.body;
123         const allowedStatuses = ['PLACED', 'PREPARING', 'DELIVERED', 'CANCELLED'];
124
125         if (!allowedStatuses.includes(status)) {
126             return res.status(400).json({ error: 'Invalid status value' });
127         }
128
129         const order = await Order.findByIdAndUpdate(
130             req.params.id,
131             { status },
132             { new: true, runValidators: true }
133         )
134             .populate('student')
135             .populate('items.menuItem');
136
137         if (!order) {
138             return res.status(404).json({ error: 'Order not found' });
139         }
140
141         res.json(order);
142     } catch (err) {
143         console.error('Error updating order status:', err.message);
144         res.status(400).json({ error: err.message });
145     }
146 });
147

```

```

148 // DELETE /orders/:id
149 router.delete('/:id', async (req, res) => {
150   try {
151     const deleted = await Order.findByIdAndDelete(req.params.id);
152
153     if (!deleted) {
154       return res.status(404).json({ error: 'Order not found' });
155     }
156
157     res.json({ message: 'Order deleted successfully' });
158   } catch (err) {
159     console.error('Error deleting order:', err.message);
160     res.status(400).json({ error: 'Invalid order ID' });
161   }
162 });
163
164 module.exports = router;

```

Step 17: Create analytics routes

```

routes > JS analytics.js > router.get('/total-spent/:studentId') callback
1  const express = require('express');
2  const mongoose = require('mongoose');
3  const Order = require('../models/Order');
4  const MenuItem = require('../models/MenuItem');
5
6  const router = express.Router();
7
8  // GET /analytics/total-spent/:studentId
9  router.get('/total-spent/:studentId', async (req, res) => {
10   try {
11     const { studentId } = req.params;
12
13     if (!mongoose.Types.ObjectId.isValid(studentId)) {
14       return res.status(400).json({ error: 'Invalid student ID' });
15     }
16
17     const result = await Order.aggregate([
18       {
19         $match: {
20           student: new mongoose.Types.ObjectId(studentId)
21         }
22       },
23       {
24         $group: {
25           _id: '$student',
26           totalSpent: { $sum: '$totalPrice' }
27         }
28       }
29     ]);
30

```

```

30
31     if (result.length === 0) {
32         return res.json({ studentId, totalSpent: 0 });
33     }
34
35     res.json({
36         studentId,
37         totalSpent: result[0].totalSpent
38     });
39 } catch (err) {
40     console.error('Error calculating total spent:', err.message);
41     res.status(500).json({ error: 'Server error' });
42 }
43 });
44

```

```

45 // GET /analytics/top-menu-items?limit=5
46 router.get('/top-menu-items', async (req, res) => {
47     try {
48         const limit = Number(req.query.limit) || 5;
49
50         const result = await Order.aggregate([
51             { $unwind: '$items' },
52             {
53                 $group: {
54                     _id: '$items.menuItem',
55                     totalQuantity: { $sum: '$items.quantity' }
56                 }
57             },
58             { $sort: { totalQuantity: -1 } },
59             { $limit: limit }
60         ]);
61
62         const formatted = [];
63
64         for (const item of result) {
65             const menuItem = await MenuItem.findById(item._id);
66             formatted.push({
67                 menuItem,
68                 totalQuantity: item.totalQuantity
69             });
70         }
71

```



```

71     res.json(formatted);
72   } catch (err) {
73     console.error('Error fetching top menu items:', err.message);
74     res.status(500).json({ error: 'Server error' });
75   }
76 });
77
78
79 // GET /analytics/daily-orders
80 router.get('/daily-orders', async (req, res) => {
81   try {
82     const result = await Order.aggregate([
83       {
84         $group: {
85           _id: {
86             $dateToString: { format: '%Y-%m-%d', date: '$createdAt' }
87           },
88           orderCount: { $sum: 1 }
89         }
90       },
91       { $sort: { _id: 1 } }
92     ]);
93

```

```

94     const formatted = result.map(item => ({
95       date: item._id,
96       orderCount: item.orderCount
97     }));
98
99     res.json(formatted);
100   } catch (err) {
101     console.error('Error fetching daily orders:', err.message);
102     res.status(500).json({ error: 'Server error' });
103   }
104 });
105
106 module.exports = router;

```

requires these analytics endpoints:

- total amount spent by a student
- top selling menu items
- daily order counts

and it specifically says to use MongoDB aggregation pipeline.

\$match : Filters documents.

\$group : Groups records and calculates totals.

\$sum : Adds values.

\$unwind : Breaks an array into separate records so each item can be counted.

\$dateToString : Converts date into YYYY-MM-DD format.

Step 18: Run server again

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\vallu\Desktop\campus-food-api_it24102099> npm start

> campus-food-api_it24102099@1.0.0 start
> node server.js

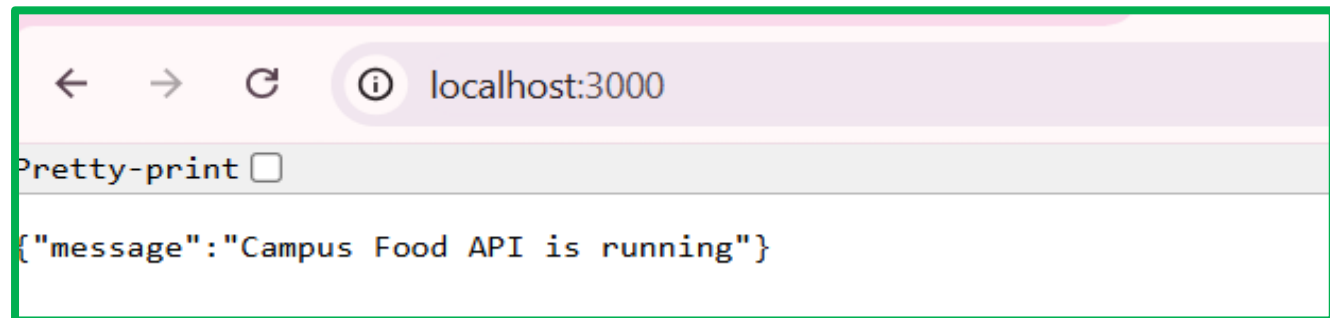
[dotenv@17.3.1] injecting env (2) from .env -- tip: 🛡️ prevent building .env in docker: https://dotenvx.com/prebuild
✅ Connected to MongoDB
🚀 Server running on http://localhost:3000
```

In Node.js:

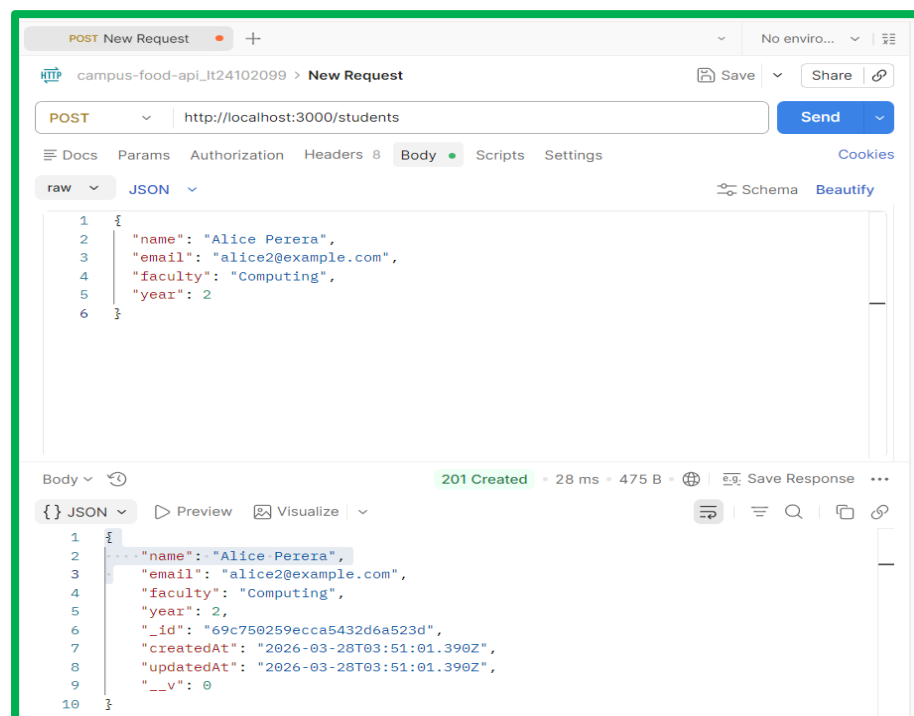
You cannot declare the same variable twice.

Step 19: Test in Postman

1. Test root route

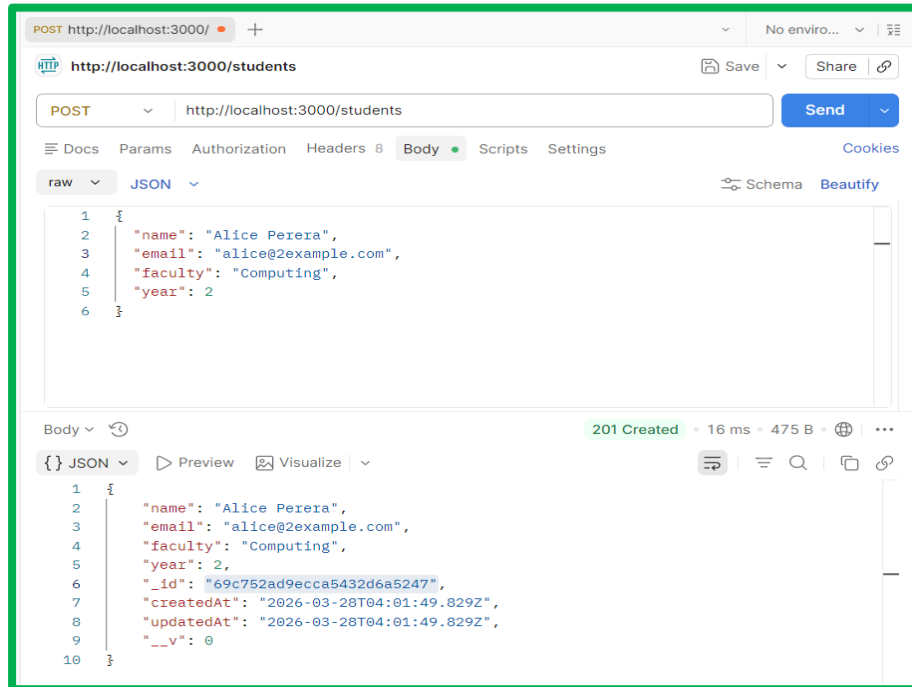


2.

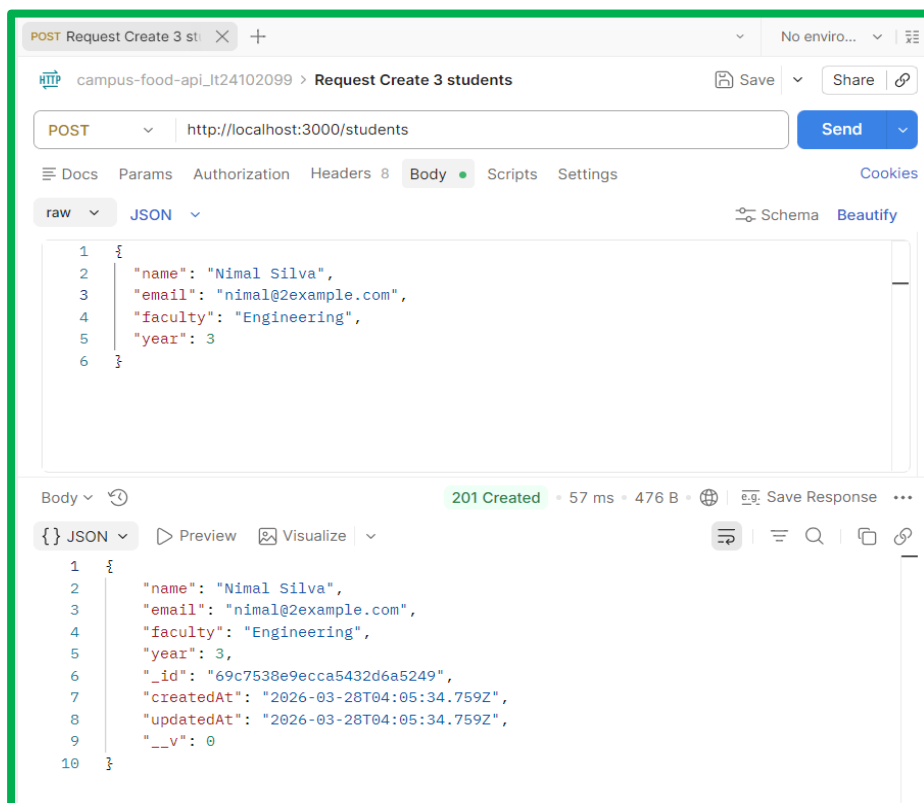


Add new collection -> Rename with a name -> Select request type (post) -> In body select raw -> Then write it. -> click send.

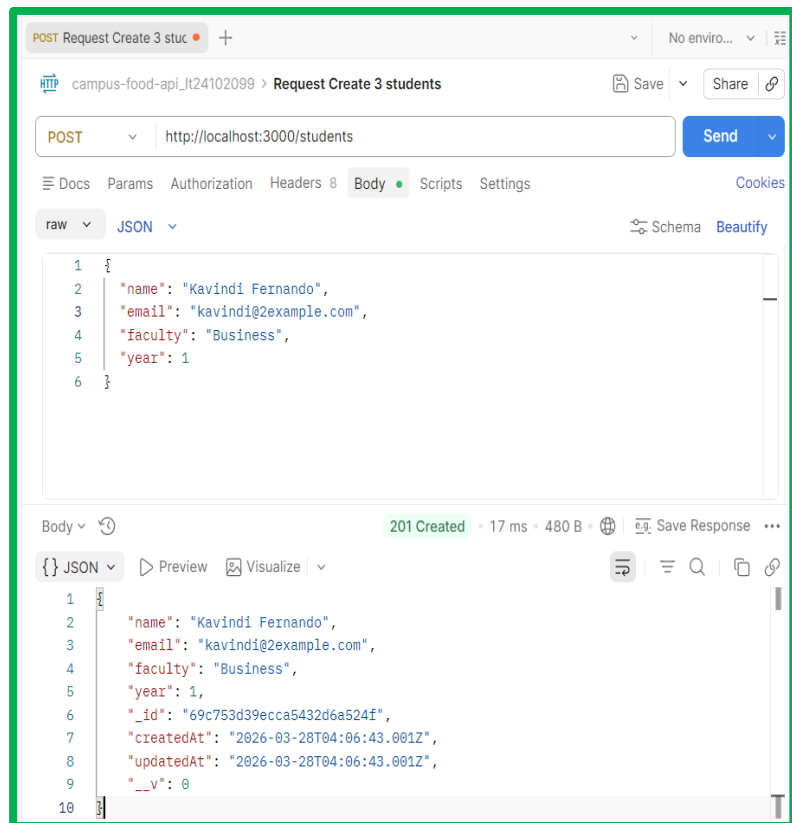
Student 1:



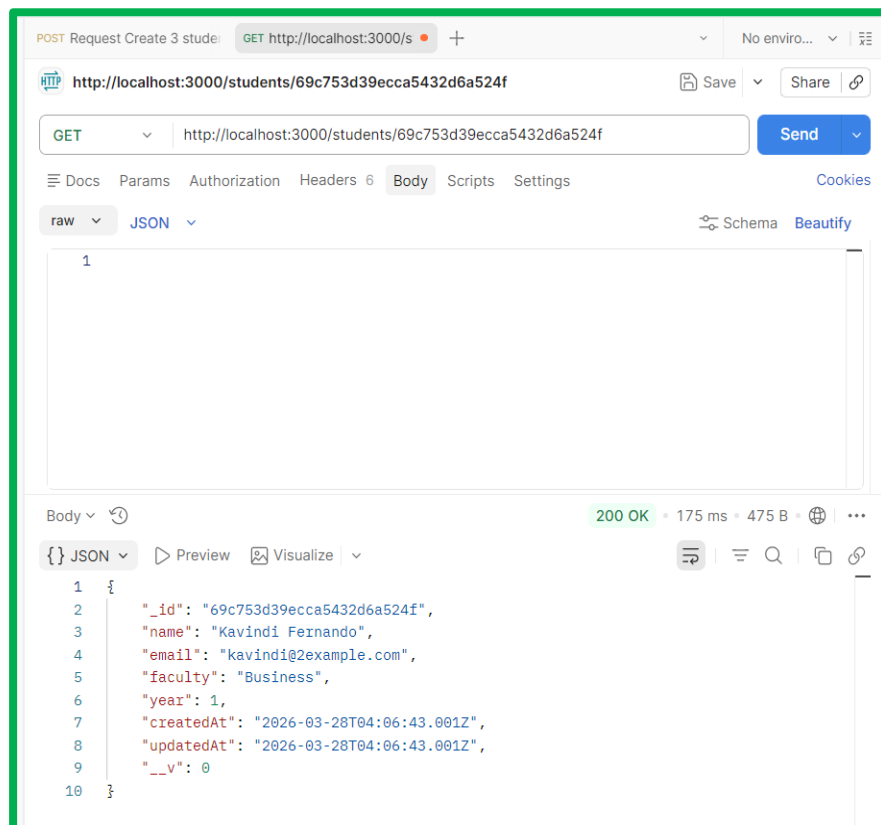
Student 2:



Student 3:



Get a student by ID



Add menu item

HTTP <http://localhost:3000/menu-items> Save Share

POST <http://localhost:3000/menu-items> Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "name": "Egg Fried Rice",
3   "price": 650,
4   "category": "Rice",
5   "isAvailable": true
6 }
```

Body JSON Preview Visualize

```
1 {
2   "name": "Egg Fried Rice",
3   "price": 650,
4   "category": "Rice",
5   "isAvailable": true,
6   "_id": "69c754899ecca5432d6a5252",
7   "createdAt": "2026-03-28T04:09:45.186Z",
8   "updatedAt": "2026-03-28T04:09:45.186Z",
9   "__v": 0
10 }
```

201 Created • 32 ms • 466 B • ...

POST Request Create 3 studer GET Get a student by ID POST <http://localhost:3000/> + No enviro... Save Share

POST <http://localhost:3000/menu-items> Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "name": "Chicken Fried Rice",
3   "price": 750,
4   "category": "Rice",
5   "isAvailable": true
6 }
```

Body JSON Preview Visualize

```
1 {
2   "name": "Chicken Fried Rice",
3   "price": 750,
4   "category": "Rice",
5   "isAvailable": true,
6   "_id": "69c754b29ecca5432d6a5254",
7   "createdAt": "2026-03-28T04:10:26.913Z",
8   "updatedAt": "2026-03-28T04:10:26.913Z",
9   "__v": 0
10 }
```

201 Created • 15 ms • 470 B • ...

HTTP <http://localhost:3000/menu-items> Save Share

POST <http://localhost:3000/menu-items> Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "name": "Coke",
3   "price": 180,
4   "category": "Beverage",
5   "isAvailable": true
6 }
```

Body Body 201 Created • 15 ms • 460 B • ...

{ JSON Preview Visualize

```
1 {
2   "name": "Coke",
3   "price": 180,
4   "category": "Beverage",
5   "isAvailable": true,
6   "_id": "69c754da9ecca5432d6a5256",
7   "createdAt": "2026-03-28T04:11:06.232Z",
8   "updatedAt": "2026-03-28T04:11:06.232Z",
9   "__v": 0
10 }
```

HTTP <http://localhost:3000/menu-items> Save Share

POST <http://localhost:3000/menu-items> Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {
2   "name": "Fish Bun",
3   "price": 100,
4   "category": "Snack",
5   "isAvailable": true
6 }
```

Body Body 201 Created • 16 ms • 461 B • ...

{ JSON Preview Visualize

```
1 {
2   "name": "Fish Bun",
3   "price": 100,
4   "category": "Snack",
5   "isAvailable": true,
6   "_id": "69c7550b9ecca5432d6a5258",
7   "createdAt": "2026-03-28T04:11:55.025Z",
8   "updatedAt": "2026-03-28T04:11:55.025Z",
9   "__v": 0
10 }
```

HTTP **http://localhost:3000/menu-items** Save Share

POST **http://localhost:3000/menu-items** Send

Docs Params Authorization Headers 8 **Body** Scripts Settings Cookies

raw JSON Schema Beautify

```

1  {
2    "name": "Tea",
3    "price": 80,
4    "category": "Beverage",
5    "isAvailable": true
6  }

```

Body 201 Created • 20 ms • 458 B

{ } JSON Preview Visualize

```

1  {
2    "name": "Tea",
3    "price": 80,
4    "category": "Beverage",
5    "isAvailable": true,
6    "_id": "69c7552e9ecca5432d6a525a",
7    "createdAt": "2026-03-28T04:12:30.123Z",
8    "updatedAt": "2026-03-28T04:12:30.123Z",
9    "__v": 0
10 }

```

List menu items

POST Request Create 3 student GET Get a student by ID **GET Create 5 menu items** No enviro...

campus-food-api_It24102099 > **Create 5 menu items** Save Share

GET **http://localhost:3000/menu-items** Send

Docs Params Authorization Headers 8 **Body** Scripts Settings Cookies

Body 200 OK • 90 ms • 1.2 KB Save Response

{ } JSON Preview Visualize

```

1  [
2    {
3      "_id": "69c7552e9ecca5432d6a525a",
4      "name": "Tea",
5      "price": 80,
6      "category": "Beverage",
7      "isAvailable": true,
8      "createdAt": "2026-03-28T04:12:30.123Z",
9      "updatedAt": "2026-03-28T04:12:30.123Z",
10     "__v": 0
11   },
12   {
13     "_id": "69c7550b9ecca5432d6a5258",
14     "name": "Fish Bun",
15     "price": 100,
16     "category": "Snack",
17     "isAvailable": true,
18     "createdAt": "2026-03-28T04:11:55.025Z",
19     "updatedAt": "2026-03-28T04:11:55.025Z",
20     "__v": 0
21   },
22   {
23     "_id": "69c754da9ecca5432d6a5256",
24     "name": "Coke",
25     "price": 180,
26     "category": "Beverage",
27     "isAvailable": true

```

Search menu items

By name:

HTTP

http://localhost:3000/menu-items/search

Save

Share

GET

http://localhost:3000/menu-items/search?name=egg

Send

Docs

Params

Authorization

Headers 6

Body

Scripts

Settings

Cookies

Query Params

Key	Value	Description	Bulk Edit	...
☒ name	egg			
Key	Value	Description		

Body

200 OK

50 ms

463 B

Visualize

JSON

Preview

Visualize

```
1  [
2    {
3      "_id": "69c75489ecca5432d6a5252",
4      "name": "Egg Fried Rice",
5      "price": 650,
6      "category": "Rice",
7      "isAvailable": true,
8      "createdAt": "2026-03-28T04:09:45.186Z",
9      "updatedAt": "2026-03-28T04:09:45.186Z",
10     "__v": 0
11   }
12 ]
```

By category:

HTTP

http://localhost:3000/menu-items/search

Save

Share

GET

http://localhost:3000/menu-items/search?category=Beverage

Send

Docs

Params

Authorization

Headers 6

Body

Scripts

Settings

Cookies

Query Params

Key	Value	Description	Bulk Edit	...
☒ category	Beverage			
Key	Value	Description		

Body

200 OK

32 ms

644 B

Visualize

JSON

Preview

Visualize

```
1  [
2    {
3      "_id": "69c754da9ecca5432d6a5256",
4      "name": "Coke",
5      "price": 180,
6      "category": "Beverage",
7      "isAvailable": true,
8      "createdAt": "2026-03-28T04:11:06.232Z",
9      "updatedAt": "2026-03-28T04:11:06.232Z",
10     "__v": 0
11   },
12   {
13     "_id": "69c7552e9ecca5432d6a525a",
14     "name": "Tea",
15     "price": 80,
16     "category": "Beverage",
17     "isAvailable": true,
18     "createdAt": "2026-03-28T04:12:30.123Z",
19     "updatedAt": "2026-03-28T04:12:30.123Z",
20     "__v": 0
21   }
22 ]
```


By both:

HTTP <http://localhost:3000/menu-items/search> Save Share

GET <http://localhost:3000/menu-items/search?category=Beverage> Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
☒ category	Beverage			
Key	Value	Description		

Body 200 OK • 19 ms • 644 B • ...

{ } JSON Preview Visualize

```
1  [
2    {
3      "_id": "69c754da9ecca5432d6a5256",
4      "name": "Coke",
5      "price": 180,
6      "category": "Beverage",
7      "isAvailable": true,
8      "createdAt": "2026-03-28T04:11:06.232Z",
9      "updatedAt": "2026-03-28T04:11:06.232Z",
10     "__v": 0
11   },
12   {
13     "_id": "69c7552e9ecca5432d6a525a",
14     "name": "Tea",
15     "price": 80,
16     "category": "Beverage",
17     "isAvailable": true,
18     "createdAt": "2026-03-28T04:12:30.123Z",
19     "updatedAt": "2026-03-28T04:12:30.123Z",
20     "__v": 0
21   }
22 ]
```

Place an order

HTTP <http://localhost:3000/orders>

POST <http://localhost:3000/orders>

Docs Params Authorization Headers 8 Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
1  {
2    "student": "69c753d39ecca5432d6a524f",
3    "items": [
4      {
5        "menuItem": "69c7550b9ecca5432d6a5258",
6        "quantity": 2
7      },
8      {
9        "menuItem": "69c754da9ecca5432d6a5256",
10       "quantity": 1
11     }
12   ]
13 }
```

HTTP <http://localhost:3000/orders> Save Share

GET <http://localhost:3000/orders?page=1&limit=2> Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
✓ page	1			
✓ limit	2			
Key	Value	Description		

Body

200 OK • 107 ms • 1.08 KB • •

JSON Preview Visualize

```

1  {
2    "page": 1,
3    "limit": 2,
4    "totalOrders": 1,
5    "totalPages": 1,
6    "orders": [
7      {
8        "_id": "69c758459ecca5432d6a5265",
9        "student": {
10         "_id": "69c753d39ecca5432d6a524f",
11         "name": "Kavindi Fernando",
12         "email": "kavindi@2example.com",
13         "faculty": "Business",
14         "year": 1,
15         "createdAt": "2026-03-28T04:06:43.001Z",
16         "updatedAt": "2026-03-28T04:06:43.001Z",
17         "__v": 0
18       },
19       "items": [
20         {

```

Get order by ID

HTTP <http://localhost:3000/orders/69c758459ecca5432d6a5265> Save Share

GET <http://localhost:3000/orders/69c758459ecca5432d6a5265> Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body

200 OK • 19 ms • 1.02 KB • •

JSON Preview Visualize

```

1  {
2    "_id": "69c758459ecca5432d6a5265",
3    "student": {
4      "_id": "69c753d39ecca5432d6a524f",
5      "name": "Kavindi Fernando",
6      "email": "kavindi@2example.com",
7      "faculty": "Business",
8      "year": 1,
9      "createdAt": "2026-03-28T04:06:43.001Z",
10     "updatedAt": "2026-03-28T04:06:43.001Z",
11     "__v": 0
12   },
13   "items": [
14     {
15       "menuItem": {
16         "_id": "69c7550b9ecca5432d6a5258",
17         "name": "Fish Bun",
18         "price": 100,
19         "category": "Snack",
20         "isAvailable": true,
21         "createdAt": "2026-03-28T04:11:55.025Z",
22         "updatedAt": "2026-03-28T04:11:55.025Z",
23         "__v": 0
24       }

```

Update order status

HTTP

http://localhost:3000/orders/69c758459ecca5432d6a5265/status

Save

Share

PATCH

http://localhost:3000/orders/69c758459ecca5432d6a5265/status

Send

Docs

Params

Authorization

Headers 8

Body

Scripts

Settings

Cookies

raw

JSON

Schema

Beautify

1 {

2 | "status": "PREPARING"

3 }

Body

200 OK

97 ms

1.02 KB

⌕

⋮

{}

JSON

Preview

Visualize

23 {

24 | "quantity": 2

25 | }

26 | }

27 | {

28 | | "menuItem": {

29 | | | "_id": "69c754da9ecca5432d6a5256",

30 | | | "name": "Coke",

31 | | | "price": 180,

32 | | | "category": "Beverage",

33 | | | "isAvailable": true,

34 | | | "createdAt": "2026-03-28T04:11:06.232Z",

35 | | | "updatedAt": "2026-03-28T04:11:06.232Z",

36 | | | "__v": 0

37 | | }

38 | | }

39 | }

40 | }

41 | "totalPrice": 380,

42 | "status": "PREPARING",

43 | "createdAt": "2026-03-28T04:25:41.229Z",

44 | "__v": 0

45 }

Delete order

HTTP

campus-food-api_It24102099 > Update order status

Save

Share

DELETE

http://localhost:3000/orders/69c758459ecca5432d6a5265

Send

Docs

Params

Authorization

Headers 8

Body

Scripts

Settings

Cookies

Query Params

Key	Value	Description	Bulk Edit	⋮
Key	Value	Description		

Body

200 OK

22 ms

307 B

⌕

Save Response

⋮

{}

JSON

Preview

Visualize

1 {

2 | "message": "Order deleted successfully"

3 }

Analytics: total spent by a student

HTTP <http://localhost:3000/analytics/total-spent/69c753d39ecca5432d6a524f> Save Share

GET <http://localhost:3000/analytics/total-spent/69c753d39ecca5432d6a524f> Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body 200 OK • 96 ms • 322 B • ...

{ } JSON Preview Visualize

```
1 {
2   "studentId": "69c753d39ecca5432d6a524f",
3   "totalSpent": 0
4 }
```

Analytics: top menu items

HTTP <http://localhost:3000/analytics/top-menu-items> Save Share

GET <http://localhost:3000/analytics/top-menu-items> Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body 200 OK • 32 ms • 267 B • ...

{ } JSON Preview Visualize

```
1 []
```

Analytics: top menu items with limit

HTTP <http://localhost:3000/analytics/top-menu-items> Save Share

GET <http://localhost:3000/analytics/top-menu-items?limit=3> Send

Docs Params Authorization Headers 6 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
☒ limit	3			
Key	Value	Description		

Body 200 OK • 14 ms • 267 B • ...

{ } JSON Preview Visualize

```
1 []
```

Analytics: daily orders

HTTP

campus-food-api_It24102099 > Analytics: top menu items with limit

Save

Share

GET

http://localhost:3000/analytics/daily-orders

Send

Docs

Params

Authorization

Headers

6

Body

Scripts

Settings

Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body

200 OK

9 ms

267 B

Save Response

JSON

Preview

Visualize

1